Name: T.Gayathri

Reg. No.: 192425239

Course Code: MLA0401

Course Name: Deep Learning

Question 1: Medical Image Classification using CNN

Answer:

A CNN is used to classify medical X-ray images into **Normal** or **Abnormal**. The image is first preprocessed and passed through convolution layers to extract features. Pooling reduces the image size while preserving important information. The flatten layer converts feature maps into a vector, and the fully connected layer classifies the image. Finally, the sigmoid layer predicts whether the image is normal or abnormal.

Workflow

Medical X-ray Image

↓

Image Preprocessing

(Resize & Normalize)

↓

Convolution Layer

(Feature Extraction)

↓

ReLU Activation

↓

Max Pooling

↓

Flatten

↓

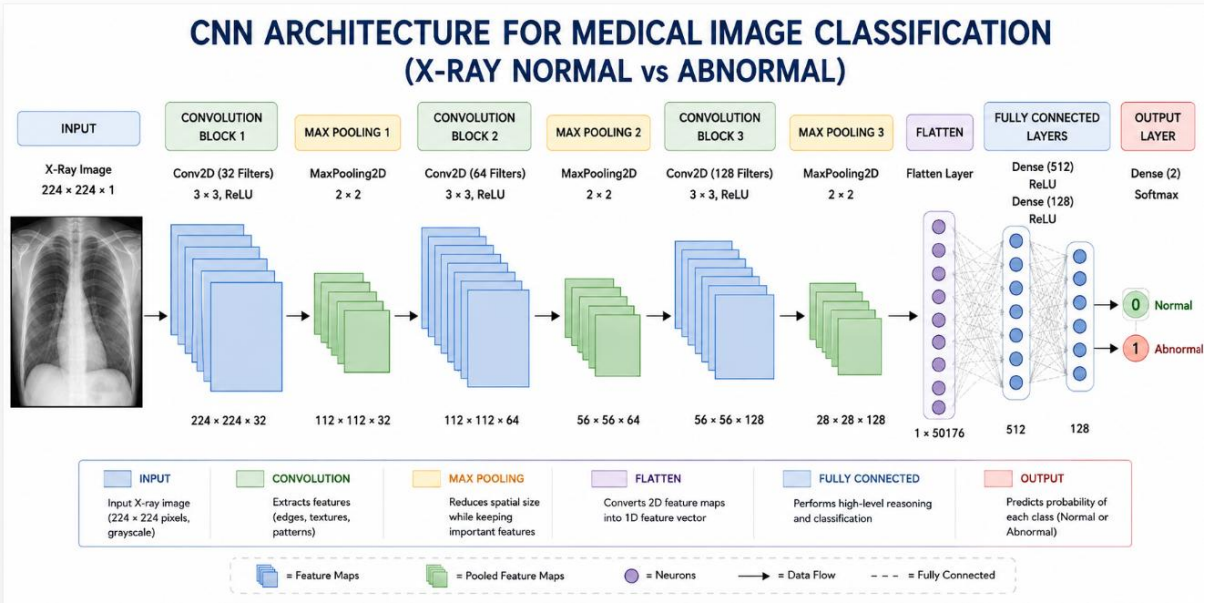
Fully Connected Layer

↓

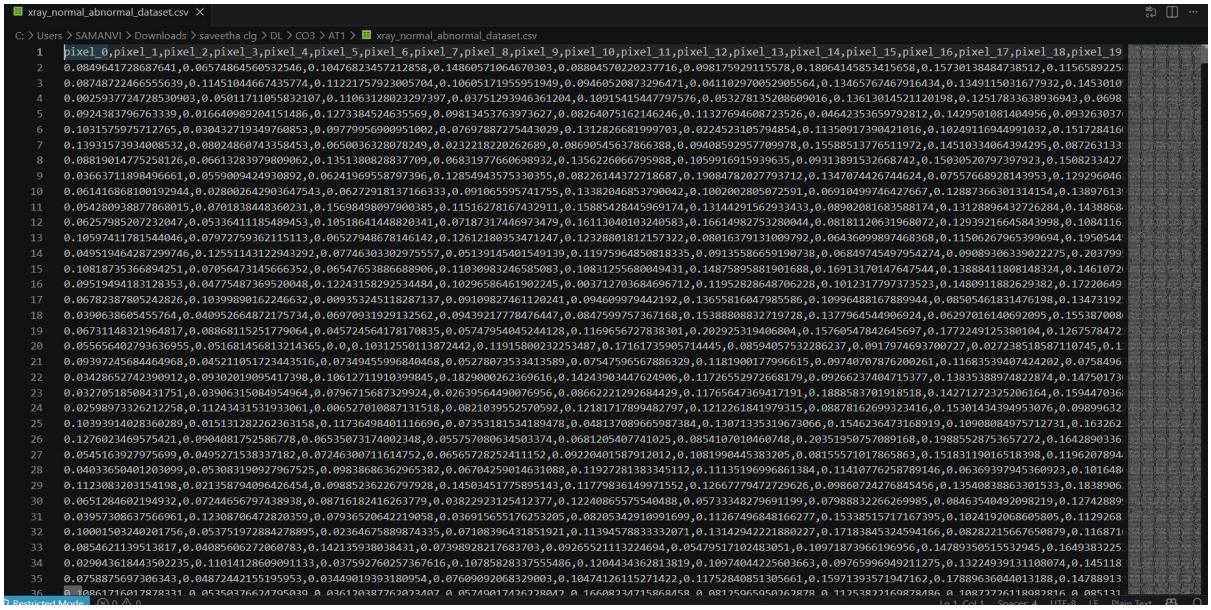
Sigmoid Output

↓

Normal / Abnormal



Datasets



==========
CNN FOR X-RAY MEDICAL IMAGE CLASSIFICATION
==========

!pip -q install tensorflow

import pandas as pd

```
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D
from tensorflow.keras.layers import Flatten, Dense

# ==========================================
# 1. UPLOAD DATASET
# ==========================================

uploaded = files.upload()

# ==========================================
# 2. LOAD DATASET
# ==========================================

df = pd.read_csv("xray_normal_abnormal_dataset.csv")

print("Dataset Shape:", df.shape)
print(df.head())

# ==========================================
# 3. SEPARATE FEATURES AND LABEL
# ==========================================

X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

# ==========================================
# 4. RESHAPE IMAGE
# ==========================================

X = X.reshape(-1, 28, 28, 1)

# ==========================================
# 5. TRAIN TEST SPLIT
# ==========================================

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# ==========================================
# 6. BUILD CNN
# ==========================================

model = Sequential()

# Convolution Block 1
model.add(
    Conv2D(
        32,
        (3,3),
        activation="relu",
        input_shape=(28,28,1)
    )
)

model.add(MaxPooling2D((2,2)))

# Convolution Block 2
model.add(
    Conv2D(
        64,
        (3,3),
        activation="relu"
    )
)

model.add(MaxPooling2D((2,2)))

# Convolution Block 3
model.add(
    Conv2D(
        128,
        (3,3),
        activation="relu"
    )
)

model.add(MaxPooling2D((2,2)))
```

```
# ==========================================
# 7. FLATTEN
# ==========================================

model.add(Flatten())

# ==========================================
# 8. FULLY CONNECTED LAYERS
# ==========================================

model.add(
    Dense(
        128,
        activation="relu"
    )
)

model.add(
    Dense(
        64,
        activation="relu"
    )
)

# ==========================================
# 9. OUTPUT LAYER
# ==========================================

model.add(
    Dense(
        2,
        activation="softmax"
    )
)

# ==========================================
# 10. COMPILE
# ==========================================

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
# Display architecture
model.summary()

# ==========================================
# 11. TRAIN MODEL
# ==========================================

history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2
)

# ==========================================
# 12. TEST MODEL
# ==========================================

loss, accuracy = model.evaluate(
    X_test,
    y_test
)

print("\nTest Accuracy:",
      round(accuracy * 100, 2), "%")

print("Test Loss:",
      round(loss, 4))

# ==========================================
# 13. ACCURACY GRAPH
# ==========================================

plt.figure(figsize=(8,5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)
```

```
plt.title("CNN Training and Validation Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.legend()
plt.grid()

plt.show()

# ==========================================
# 14. LOSS GRAPH
# ==========================================

plt.figure(figsize=(8,5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title("CNN Training and Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.legend()
plt.grid()

plt.show()

# ==========================================
# 15. PREDICTION
# ==========================================

predictions = model.predict(X_test)

predicted_classes = np.argmax(
    predictions,
    axis=1
)
```

```
# ==========================================
# 16. CONFUSION MATRIX
# ==========================================

cm = confusion_matrix(
    y_test,
    predicted_classes
)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Normal", "Abnormal"]
)

disp.plot()

plt.title("CNN Confusion Matrix")

plt.show()

# ==========================================
# 17. SAMPLE PREDICTION
# ==========================================

sample = X_test[0]

prediction = model.predict(
    sample.reshape(1,28,28,1)
)

result = np.argmax(prediction)

if result == 0:
    print("Prediction: NORMAL X-RAY")
else:
    print("Prediction: ABNORMAL X-RAY")
```

Output:

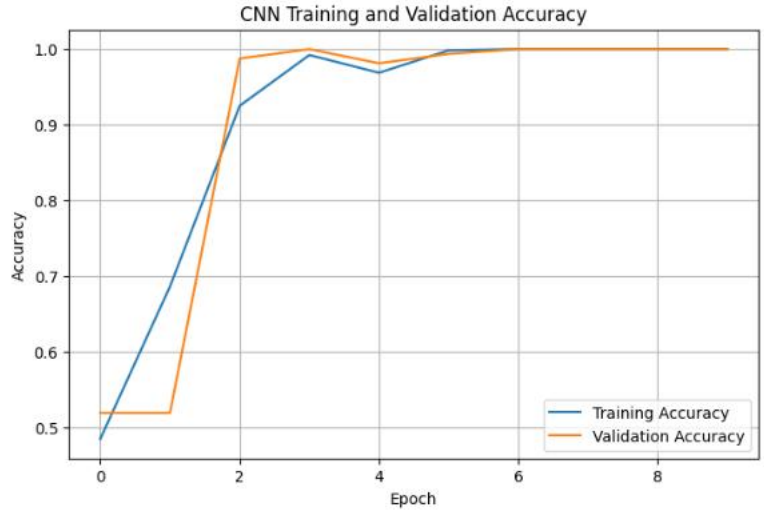
Choose Files xray_norma...dataset.csv

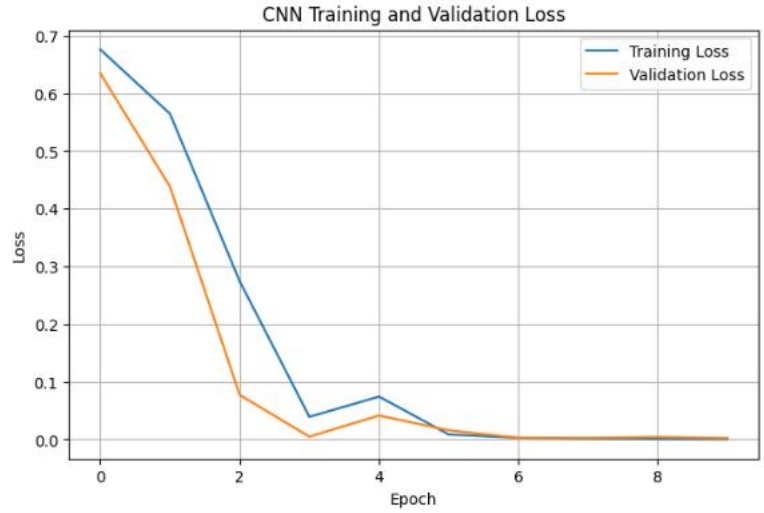
xray_normal_abnormal_dataset.csv(text/csv) - 15286512 bytes, last modified: 8/10/2026 - 100% done

```
Saving xray_normal_abnormal_dataset.csv to xray_normal_abnormal_dataset.csv
Dataset Shape: (1000, 785)
    pixel_0   pixel_1   pixel_2   pixel_3   pixel_4   pixel_5   pixel_6  \
0  0.084964  0.065749  0.104768  0.148606  0.088046  0.098176  0.180641
1  0.087487  0.114510  0.112218  0.106052  0.094605  0.041103  0.134658
2  0.002594  0.050117  0.110631  0.037513  0.109154  0.053278  0.136130
3  0.092438  0.016641  0.127338  0.098135  0.082641  0.113277  0.046424
4  0.103158  0.030433  0.097800  0.076979  0.131283  0.022452  0.113509

    pixel_7   pixel_8   pixel_9  ...  pixel_775  pixel_776  pixel_777  \
0  0.157301  0.115659  0.162375  ...   0.091458   0.115508   0.123417
1  0.134912  0.145301  0.089602  ...   0.143422   0.143739   0.103429
2  0.125178  0.069833  0.187262  ...   0.133225   0.058460   0.177204
3  0.142950  0.093263  0.086566  ...   0.206999   0.147501   0.116858
4  0.102491  0.151728  0.159474  ...   0.163008   0.184972   0.099520

   pixel_778  pixel_779  pixel_780  pixel_781  pixel_782  pixel_783  label
0   0.118077   0.133373   0.103642   0.032224   0.091484   0.093064    0.0
1   0.024085   0.143908   0.126008   0.074385   0.078956   0.089278    0.0
2   0.101901   0.074035   0.124273   0.069958   0.118674   0.093321    0.0
3   0.117485   0.107043   0.027286   0.145436   0.077929   0.086361    0.0
4   0.061094   0.104872   0.093037   0.107571   0.114260   0.066264    0.0

[5 rows x 785 columns]
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_c
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_2 (Conv2D)	(None, 3, 3, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 1, 1, 128)	0
flatten (Flatten)	(None, 128)	0
dense (Dense)	(None, 128)	16,512
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 2)	130

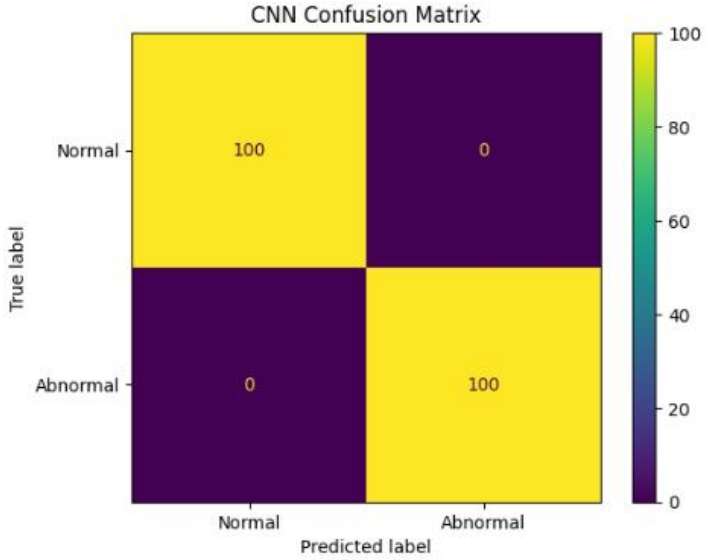
Total params: 117,570 (459.26 KB)
Trainable params: 117,570 (459.26 KB)





0 2 4 6 8
Epoch

```
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step
```




```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
Prediction: ABNORMAL X-RAY
```

Question 2: Face Recognition System using CNN

Answer:CNN is used to recognize a person's face by extracting facial features such as eyes, nose, and mouth. Pooling reduces the feature size, and the fully connected layer compares the extracted features with stored faces. The Softmax layer predicts the correct person's identity.

Workflow

Face Image

↓

Image Preprocessing

↓

Convolution Layer

↓

Feature Extraction

↓

Max Pooling

↓

Flatten

↓

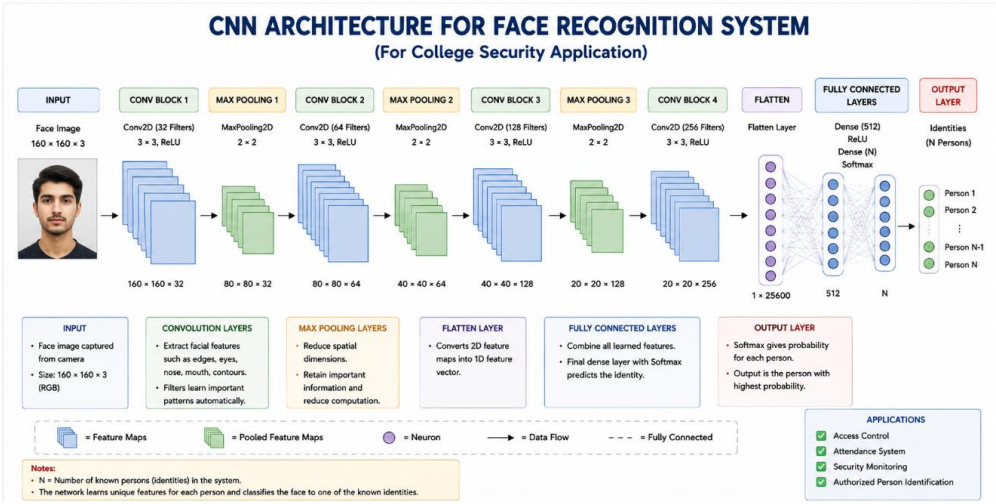
Fully Connected Layer

↓

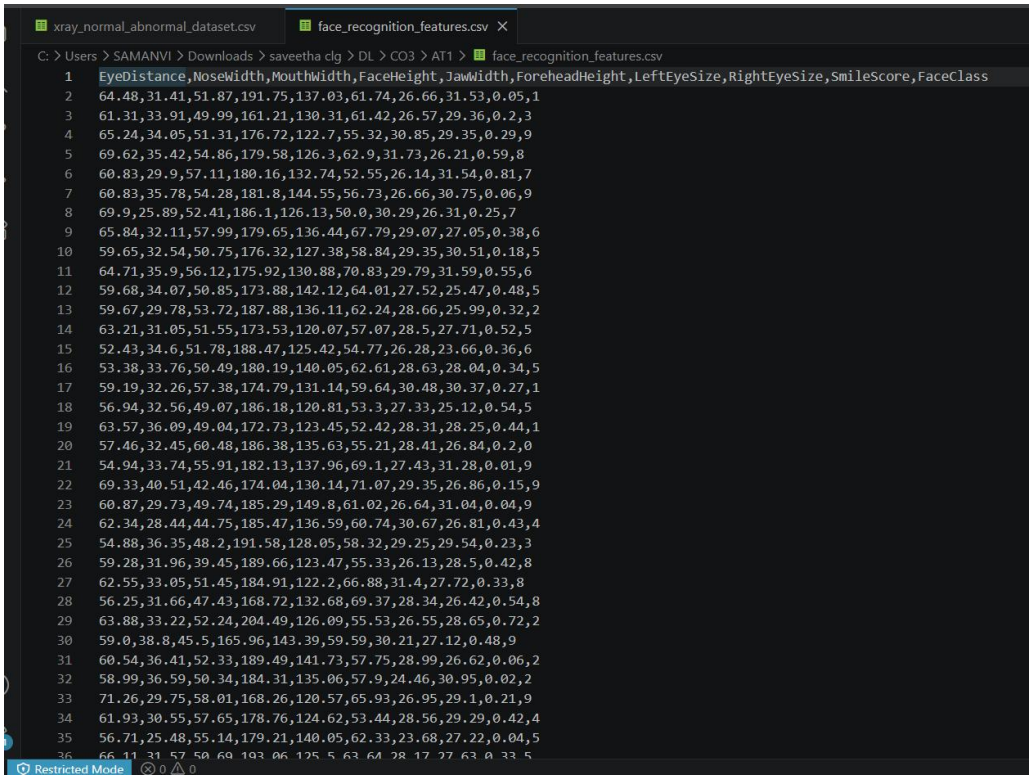
Softmax Output

↓

Recognized Person



Dataset



Program:

```
# ============================================
# FACE RECOGNITION USING DEEP NEURAL NETWORK
# ============================================

!pip -q install tensorflow
```

```
# ----------------------------------------
# Import Libraries
# ----------------------------------------

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# ----------------------------------------
# Upload Dataset
# ----------------------------------------

uploaded = files.upload()

# ----------------------------------------
# Read Dataset
# ----------------------------------------

df = pd.read_csv("face_recognition_features.csv")

print("Dataset Shape:", df.shape)

print("\nFirst 5 Rows:")
print(df.head())

# ----------------------------------------
# Separate Features and Label
# ----------------------------------------

X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

# ----------------------------------------
# Train-Test Split
# ----------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# ----------------------------------------
# Normalize Features
# ----------------------------------------

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

# ----------------------------------------
# Build Deep Neural Network
# ----------------------------------------

model = Sequential()

# Input + Hidden Layer 1
model.add(
    Dense(
        64,
        activation="relu",
        input_shape=(X_train.shape[1],)
    )
)

# Hidden Layer 2
model.add(
    Dense(
        128,
        activation="relu"
    )
)

# Hidden Layer 3
model.add(
    Dense(
        64,
        activation="relu"
    )
```

```
)

# Hidden Layer 4
model.add(
    Dense(
        32,
        activation="relu"
    )
)

# Output Layer
model.add(
    Dense(
        10,
        activation="softmax"
    )
)

# ----------------------------------------
# Compile Model
# ----------------------------------------

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Show Architecture
model.summary()

# ----------------------------------------
# Train Model
# ----------------------------------------

history = model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.20,
    verbose=1
)

# ----------------------------------------
# Evaluate Model
```

```
# ----------------------------------------

loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("\nTest Accuracy:",
      round(accuracy * 100, 2), "%")

print("Test Loss:",
      round(loss, 4))

# ==========================================
# GRAPH 1: ACCURACY
# ==========================================

plt.figure(figsize=(8,5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title("Face Recognition Model Accuracy")

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.legend()

plt.grid()

plt.show()

# ==========================================
# GRAPH 2: LOSS
# ==========================================
```

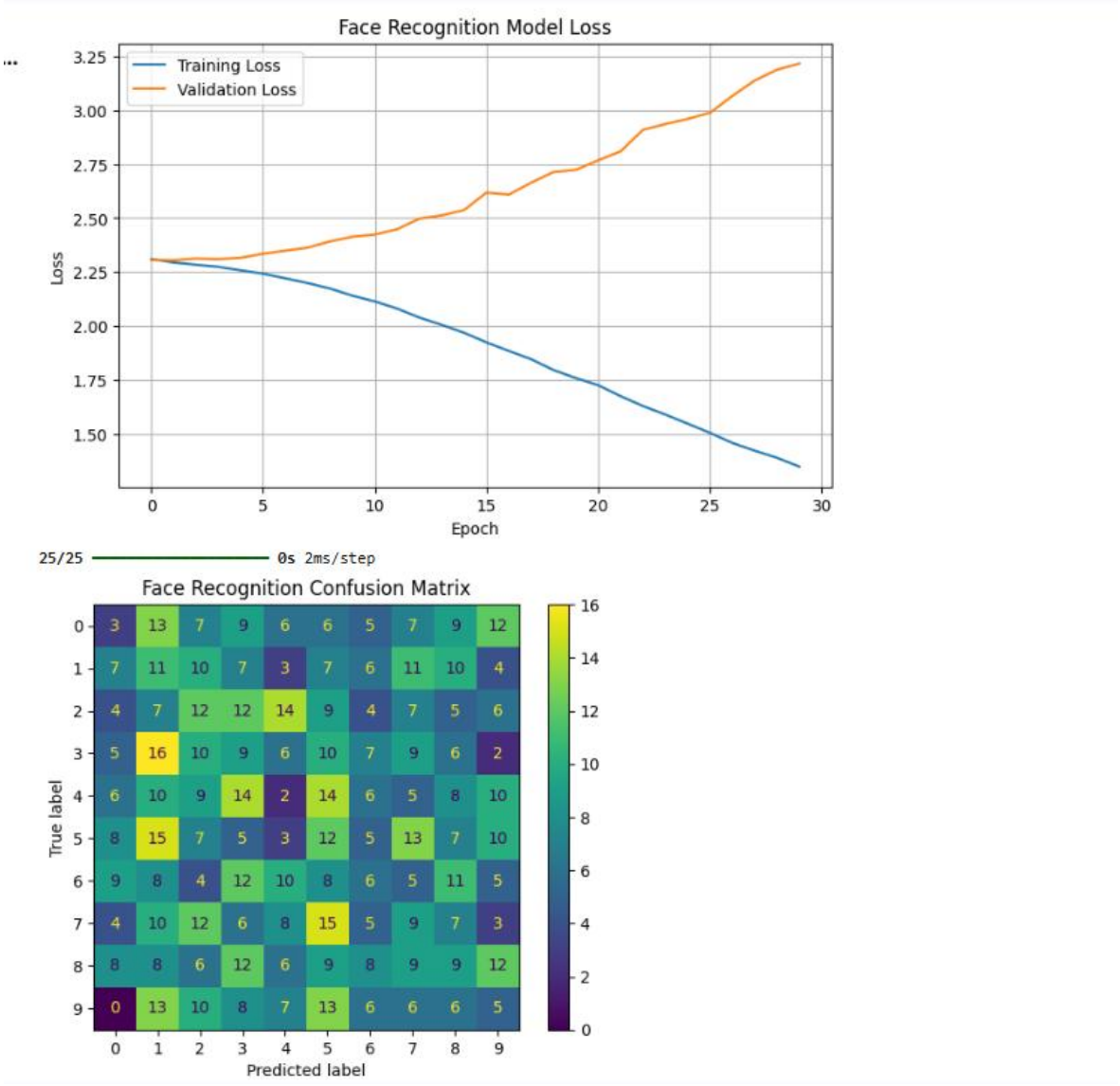
```
plt.figure(figsize=(8,5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)
plt.title("Face Recognition Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid()
plt.show()
predictions = model.predict(X_test)
predicted_classes = np.argmax(
    predictions,
    axis=1
)
# ==========================================
# GRAPH 3: CONFUSION MATRIX
# ==========================================
cm = confusion_matrix(
    y_test,
    predicted_classes
)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm
)
disp.plot()
plt.title("Face Recognition Confusion Matrix")
plt.show()
# ==========================================
# SAMPLE PREDICTION
# ==========================================

sample = X_test[0].reshape(1, -1)

prediction = model.predict(sample)

person = np.argmax(prediction)

print("\nPredicted Person/Class:", person)
```

```
print(
    "Confidence:",
    round(np.max(prediction) * 100, 2),
    "%"
)
```

```
Choose Files face_recog...features.csv
face_recognition_features.csv(text/csv) - 224473 bytes, last modified: 8/10/2026 - 100% done
Saving face_recognition_features.csv to face_recognition_features.csv
Dataset Shape: (4000, 10)

First 5 Rows:
   EyeDistance  NoseWidth  MouthWidth  FaceHeight  JawWidth  ForeheadHeight  \
0        64.48      31.41       51.87      191.75    137.03           61.74
1        61.31      33.91       49.99      161.21    130.31           61.42
2        65.24      34.05       51.31      176.72    122.70           55.32
3        69.62      35.42       54.86      179.58    126.30           62.90
4        60.83      29.90       57.11      180.16    132.74           52.55

   LeftEyeSize  RightEyeSize  SmileScore  FaceClass
0        26.66         31.53        0.05          1
1        26.57         29.36        0.20          3
2        30.85         29.35        0.29          9
3        31.73         26.21        0.59          8
4        26.14         31.54        0.81          7
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 64)	640
dense_4 (Dense)	(None, 128)	8,320
dense_5 (Dense)	(None, 64)	8,256
dense_6 (Dense)	(None, 32)	2,080
dense_7 (Dense)	(None, 10)	330

```
 Total params: 19,626 (76.66 KB)
 Trainable params: 19,626 (76.66 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/30
80/80 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - accuracy: 0.0918 - loss: 2.3104 - val_accuracy: 0.0766 - val_loss: 2.3066
Epoch 2/30
80/80 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.1289 - loss: 2.2943 - val_accuracy: 0.1203 - val_loss: 2.3051
Epoch 3/30
80/80 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.1441 - loss: 2.2841 - val_accuracy: 0.0969 - val_loss: 2.3126
Epoch 4/30
80/80 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.1523 - loss: 2.2742 - val_accuracy: 0.1047 - val_loss: 2.3101
Epoch 5/30
80/80 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.1602 - loss: 2.2580 - val_accuracy: 0.1125 - val_loss: 2.3162
Epoch 6/30
80/80 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.1820 - loss: 2.2427 - val_accuracy: 0.0906 - val_loss: 2.3347
Epoch 7/30
80/80 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.1809 - loss: 2.2211 - val_accuracy: 0.0781 - val_loss: 2.3490
Epoch 8/30
```



3.Autonomous Vehicles — Object Detection using CNN

Answer:
The vehicle camera captures road images. CNN extracts important features like edges and shapes. Object detection identifies vehicles, pedestrians, and traffic signs. The system classifies each object and helps the vehicle make safe driving decisions.

Workflow
Camera Image
↓
Image Preprocessing
↓
Convolution Layers
↓

Feature Extraction

↓

Pooling

↓

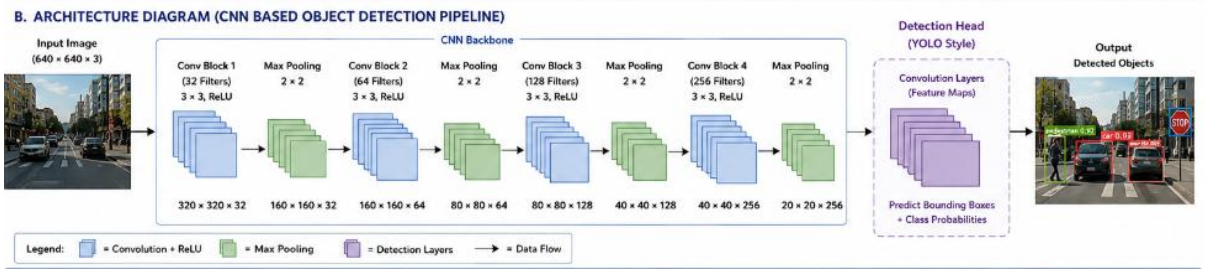
Object Detection

↓

Classification

↓

Car / Person / Traffic Sign



Dataset:

xray_normal_abnormal_dataset.csv | autonomous_vehicle_object_detection.csv | face_recognition_features.csv

C: > Users > SAMANVI > Downloads > saveetha clg > DL > CO3 > AT1 > autonomous_vehicle_object_detection.csv

```
image_id,object_id,class,xmin,ymin,xmax,ymax
img_0001.jpg,1,pedestrian,270,106,371,196
img_0001.jpg,2,pedestrian,121,214,225,318
img_0001.jpg,3,road_sign,99,359,152,391
img_0002.jpg,1,car,343,293,374,386
img_0002.jpg,2,road_sign,160,313,211,450
img_0003.jpg,1,car,58,169,179,258
img_0004.jpg,1,car,445,174,536,254
img_0004.jpg,2,car,243,319,275,449
img_0004.jpg,3,pedestrian,20,328,88,375
img_0004.jpg,4,road_sign,315,13,458,51
img_0005.jpg,1,car,339,91,479,180
img_0005.jpg,2,car,263,34,370,144
img_0006.jpg,1,car,387,1,422,84
img_0006.jpg,2,car,309,190,356,309
img_0006.jpg,3,car,201,269,325,346
img_0006.jpg,4,car,461,214,552,283
img_0007.jpg,1,car,337,366,419,419
img_0008.jpg,1,road_sign,187,379,325,449
img_0008.jpg,2,car,300,64,418,164
img_0009.jpg,1,pedestrian,491,135,608,227
img_0010.jpg,1,car,391,162,455,224
img_0010.jpg,2,pedestrian,489,230,559,287
img_0010.jpg,3,road_sign,327,267,390,329
img_0011.jpg,1,car,61,215,127,343
img_0011.jpg,2,pedestrian,213,34,307,162
img_0011.jpg,3,car,461,130,491,164
img_0011.jpg,4,pedestrian,254,397,386,453
img_0012.jpg,1,pedestrian,345,41,451,121
img_0013.jpg,1,road_sign,240,51,365,84
img_0013.jpg,2,road_sign,406,230,544,274
img_0013.jpg,3,pedestrian,35,12,96,112
img_0014.jpg,1,pedestrian,283,65,354,139
img_0014.jpg,2,car,133,283,190,420
img_0014.jpg,3,car,285,330,433,451
img_0015.jpg,1,car,224,384,280,475
```

Program:

```
# ============================================
# AUTONOMOUS VEHICLE OBJECT CLASSIFICATION
# USING CNN
# ============================================

!pip -q install tensorflow

# ----------------------------------------
# Import Libraries
# ----------------------------------------

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

# ----------------------------------------
# Upload Dataset
# ----------------------------------------

uploaded = files.upload()

# ----------------------------------------
# Load Dataset
# ----------------------------------------

df = pd.read_csv(
    "autonomous_vehicle_object_detection.csv"
)

print("Dataset Shape:", df.shape)

print("\nFirst 5 Rows:")
print(df.head())

# ----------------------------------------
# Prepare Features
```

```
# ----------------------------------------

X = df[
    ["xmin", "ymin", "xmax", "ymax"]
].values

y = df["class"].values

# ----------------------------------------
# Encode Classes
# ----------------------------------------

encoder = LabelEncoder()

y = encoder.fit_transform(y)

print("\nClasses:")
print(encoder.classes_)

# ----------------------------------------
# Split Dataset
# ----------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# ----------------------------------------
# Normalize Coordinates
# ----------------------------------------

X_train = X_train / 640.0
X_test = X_test / 640.0

# ----------------------------------------
# Build Neural Network
# ----------------------------------------

model = Sequential()

model.add(
    Dense(
```

```
        64,
        activation="relu",
        input_shape=(4,)
    )
)

model.add(
    Dense(
        128,
        activation="relu"
    )
)

model.add(
    Dropout(0.2)
)

model.add(
    Dense(
        64,
        activation="relu"
    )
)

model.add(
    Dense(
        32,
        activation="relu"
    )
)

# Output:
# Car, Pedestrian, Road Sign

model.add(
    Dense(
        3,
        activation="softmax"
    )
)

# ----------------------------------------
# Compile
# ----------------------------------------

model.compile(
```

```
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

# ----------------------------------------
# Train
# ----------------------------------------

history = model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.20,
    verbose=1
)

# ----------------------------------------
# Test
# ----------------------------------------

loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print(
    "\nTest Accuracy:",
    round(accuracy * 100, 2),
    "%"
)

# ========================================
# GRAPH 1 — ACCURACY
# ========================================

plt.figure(figsize=(8,5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)
```

```
plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title(
    "Autonomous Vehicle Object Classification Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.legend()
plt.grid()

plt.show()

# ==========================================
# GRAPH 2 — LOSS
# ==========================================

plt.figure(figsize=(8,5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title(
    "Autonomous Vehicle Model Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.legend()
plt.grid()

plt.show()
```

```
# ==========================================
# PREDICTION
# ==========================================

predictions = model.predict(X_test)

predicted_classes = np.argmax(
    predictions,
    axis=1
)

# ------------------------------------------
# Display Sample Predictions
# ------------------------------------------

for i in range(10):

    actual = encoder.inverse_transform(
        [y_test[i]]
    )[0]

    predicted = encoder.inverse_transform(
        [predicted_classes[i]]
    )[0]

    confidence = np.max(
        predictions[i]
    ) * 100

    print(
        "Actual:",
        actual,
        "| Predicted:",
        predicted,
        "| Confidence:",
        round(confidence, 2),
        "%"
    )
```

Choose Files autonomou...ection.csv

autonomous_vehicle_object_detection.csv(text/csv) - 91243 bytes, last modified: 8/10/2026 - 100% done

Saving autonomous_vehicle_object_detection.csv to autonomous_vehicle_object_detection.csv

Dataset Shape: (2433, 7)

First 5 Rows:

	image_id	object_id	class	xmin	ymin	xmax	ymax
0	img_0001.jpg	1	pedestrian	270	106	371	196
1	img_0001.jpg	2	pedestrian	121	214	225	318
2	img_0001.jpg	3	road_sign	99	359	152	391
3	img_0002.jpg	1	car	343	293	374	386
4	img_0002.jpg	2	road_sign	160	313	211	450

Classes:

['car' 'pedestrian' 'road_sign']

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequentia

  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_8 (Dense)	(None, 64)	320
dense_9 (Dense)	(None, 128)	8,320
dropout (Dropout)	(None, 128)	0
dense_10 (Dense)	(None, 64)	8,256
dense_11 (Dense)	(None, 32)	2,080
dense_12 (Dense)	(None, 3)	99

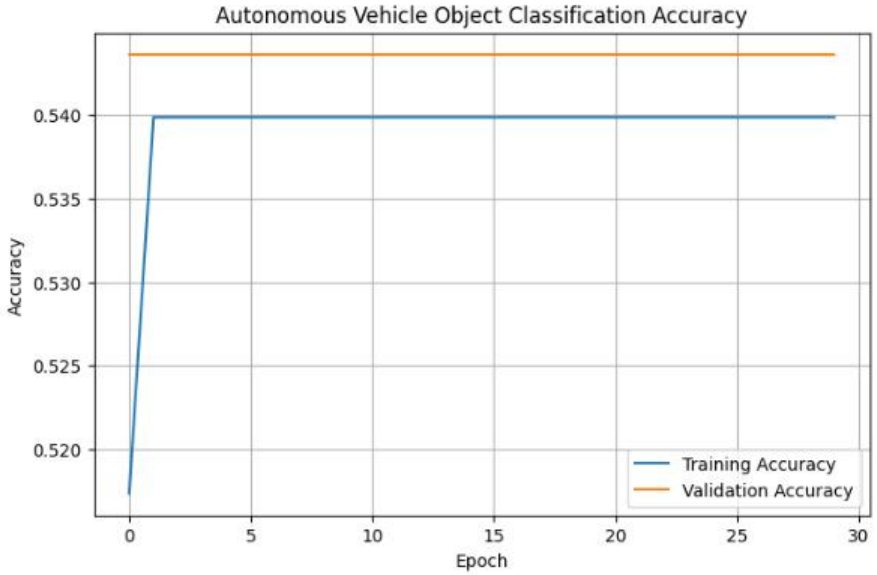
Total params: 19,075 (74.51 KB)

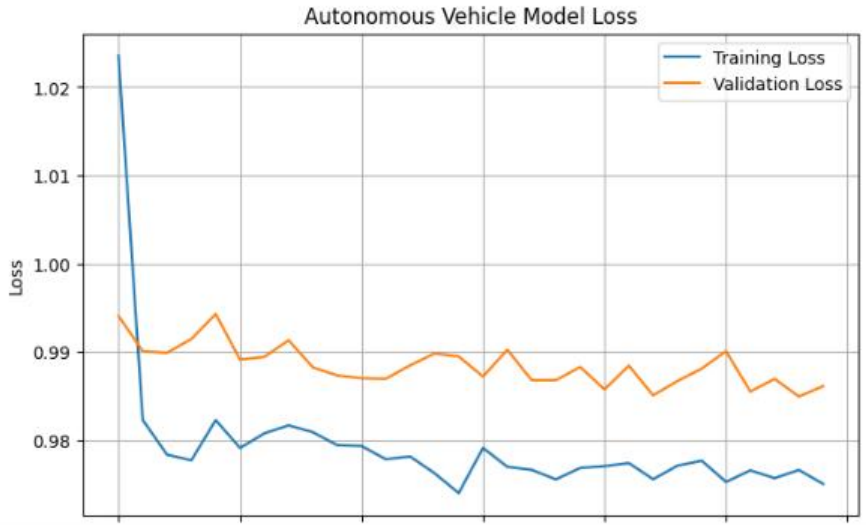
Trainable params: 19,075 (74.51 KB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/30
49/49 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.5174 - loss: 1.0236 - val_accuracy: 0.5436 - val_loss: 0.9941
Epoch 2/30
49/49 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.5398 - loss: 0.9823 - val_accuracy: 0.5436 - val_loss: 0.9901
Epoch 3/30
49/49 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.5398 - loss: 0.9784 - val_accuracy: 0.5436 - val_loss: 0.9899
Epoch 4/30
49/49 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.5398 - loss: 0.9777 - val_accuracy: 0.5436 - val_loss: 0.9915
Epoch 5/30
49/49 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.5398 - loss: 0.9823 - val_accuracy: 0.5436 - val_loss: 0.9943
Epoch 6/30
49/49 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.5398 - loss: 0.9791 - val_accuracy: 0.5436 - val_loss: 0.9891
Epoch 7/30
49/49 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.5398 - loss: 0.9808 - val_accuracy: 0.5436 - val_loss: 0.9894
Epoch 8/30
49/49 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.5398 - loss: 0.9817 - val_accuracy: 0.5436 - val_loss: 0.9913
Epoch 9/30
49/49 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.5398 - loss: 0.9809 - val_accuracy: 0.5436 - val_loss: 0.9883
```

```
Test Accuracy: 54.0 %
```





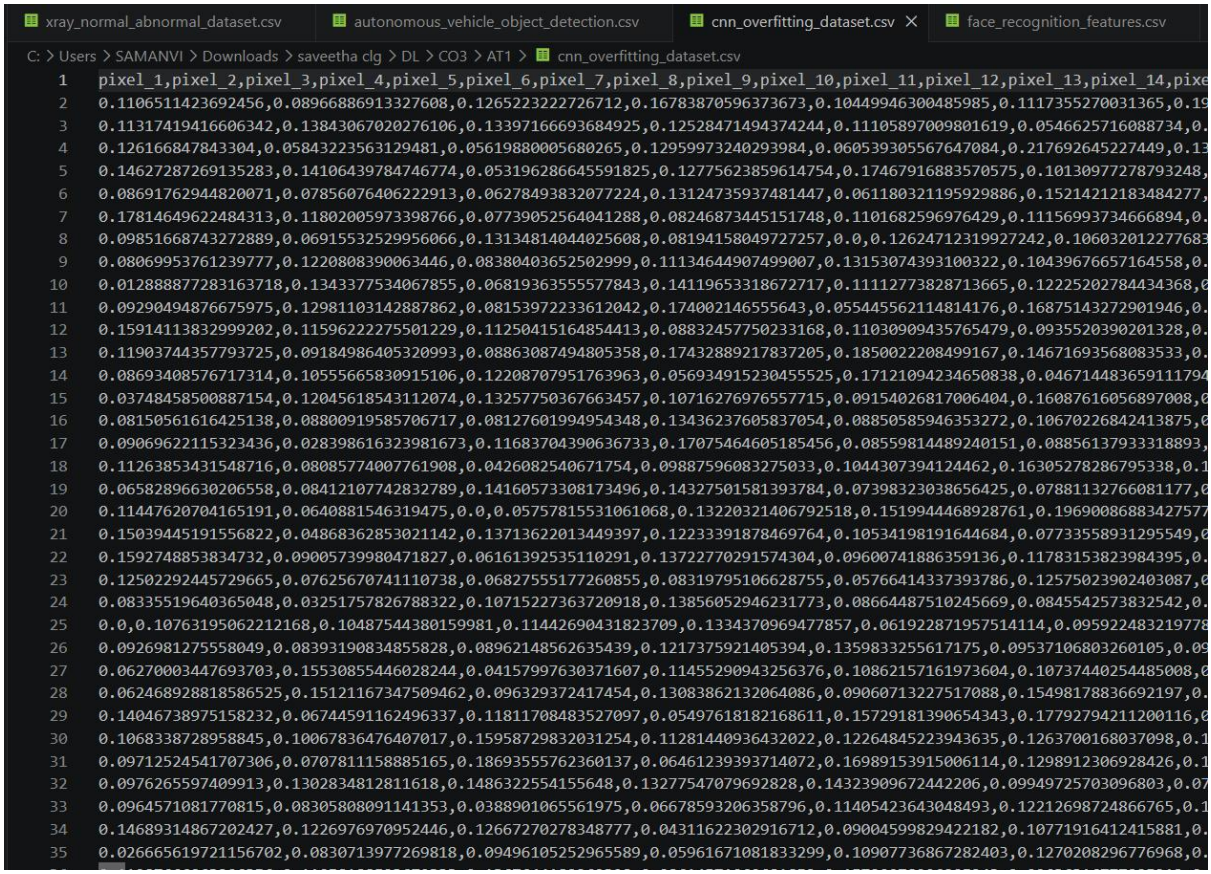
4. Case Study: CNN Overfitting

Answer:
Overfitting occurs when the CNN learns the training data too well but performs poorly on new data. It is reduced using **Dropout**, **Batch Normalization**, and **Data Augmentation**, which improve the model's ability to generalize.

Workflow

Training Images
↓
CNN Training
↓
High Training Accuracy
(98%)
↓
Testing Images
↓
Low Testing Accuracy
(75%)
↓
Overfitting
↓
Dropout
Batch Normalization
Data Augmentation
↓
Improved Accuracy

Dataset:



Program:

```
# ============================================
# CNN OVERFITTING - SOLUTION
# Dropout + Batch Normalization
# + Data Augmentation
# ============================================

!pip -q install tensorflow

# ----------------------------------------
# Import Libraries
# ----------------------------------------

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files

from sklearn.model_selection import train_test_split
```

```
import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import (
    Conv2D,
    MaxPooling2D,
    BatchNormalization,
    Flatten,
    Dense,
    Dropout
)

from tensorflow.keras.preprocessing.image import ImageDataGenerator

# ----------------------------------------
# Upload Dataset
# ----------------------------------------

uploaded = files.upload()

# ----------------------------------------
# Load Dataset
# ----------------------------------------

df = pd.read_csv("cnn_overfitting_dataset.csv")

print("Dataset Shape:", df.shape)

print(df.head())

# ----------------------------------------
# Separate Features and Labels
# ----------------------------------------

X = df.iloc[:, :-1].values

y = df["label"].values

# ----------------------------------------
# Reshape into 28 × 28 Images
# ----------------------------------------

X = X.reshape(-1, 28, 28, 1)

# ----------------------------------------
```

```
# Train-Test Split
# ----------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))

# ============================================
# DATA AUGMENTATION
# ============================================

datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.10,
    height_shift_range=0.10,
    zoom_range=0.10,
    horizontal_flip=True
)

datagen.fit(X_train)

# ============================================
# BUILD CNN WITH REGULARIZATION
# ============================================

model = Sequential()

# ----------------------------------------
# CNN BLOCK 1
# ----------------------------------------

model.add(
    Conv2D(
        32,
        (3,3),
        activation="relu",
        padding="same",
        input_shape=(28,28,1)
    )
```

```
)

# Batch Normalization
model.add(
    BatchNormalization()
)

# Pooling
model.add(
    MaxPooling2D((2,2))
)

# ----------------------------------------
# CNN BLOCK 2
# ----------------------------------------

model.add(
    Conv2D(
        64,
        (3,3),
        activation="relu",
        padding="same"
    )
)

# Batch Normalization
model.add(
    BatchNormalization()
)

# Pooling
model.add(
    MaxPooling2D((2,2))
)

# ----------------------------------------
# CNN BLOCK 3
# ----------------------------------------

model.add(
    Conv2D(
        128,
        (3,3),
        activation="relu",
        padding="same"
    )
```

```
)

# Batch Normalization
model.add(
    BatchNormalization()
)

# Pooling
model.add(
    MaxPooling2D((2,2))
)

# ----------------------------------------
# Flatten
# ----------------------------------------

model.add(
    Flatten()
)

# ----------------------------------------
# Fully Connected Layer
# ----------------------------------------

model.add(
    Dense(
        128,
        activation="relu"
    )
)

# ----------------------------------------
# DROPOUT
# ----------------------------------------

model.add(
    Dropout(0.5)
)

# ----------------------------------------
# Output
# ----------------------------------------

model.add(
    Dense(
        1,
```

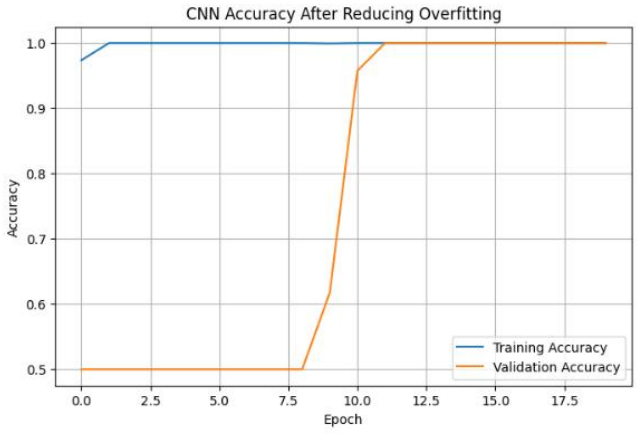
```
        activation="sigmoid"
    )
)

# ==========================================
# COMPILE MODEL
# ==========================================

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

# ----------------------------------------
# Display Architecture
# ----------------------------------------

model.summary()

# ==========================================
# TRAIN MODEL
# ==========================================

history = model.fit(
    datagen.flow(
        X_train,
        y_train,
        batch_size=32
    ),
    epochs=20,
    validation_data=(X_test, y_test)
)

# ==========================================
# EVALUATION
# ==========================================

loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("\nFinal Test Accuracy:",
      round(accuracy * 100, 2), "%")
```

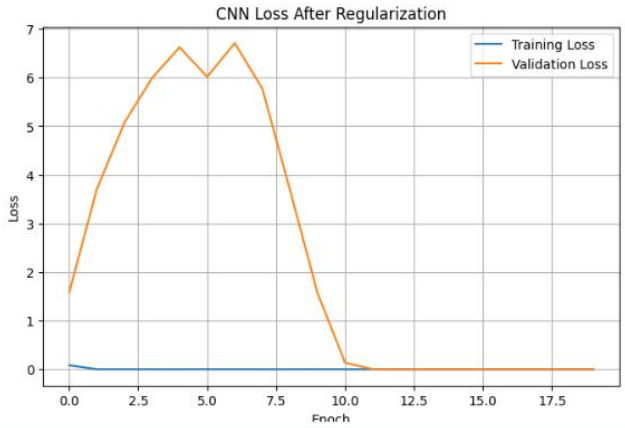
```
print("Final Test Loss:",
      round(loss, 4))

# ==========================================
# GRAPH 1
# ACCURACY
# ==========================================

plt.figure(figsize=(8,5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title(
    "CNN Accuracy After Reducing Overfitting"
)

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.legend()

plt.grid()

plt.show()

# ==========================================
# GRAPH 2
# LOSS
# ==========================================

plt.figure(figsize=(8,5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)
```

```
plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title(
    "CNN Loss After Regularization"
)

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.legend()

plt.grid()

plt.show()

# ============================================
# PREDICTION
# ============================================

predictions = model.predict(X_test)

predictions = (
    predictions > 0.5
).astype(int)

print("\nSample Predictions:")

for i in range(10):

    actual = int(y_test[i])

    predicted = int(predictions[i][0])

    print(
        "Actual:",
        actual,
        "Predicted:",
        predicted
    )
```

```
    pixel_8   pixel_9  pixel_10  ...  pixel_776  pixel_777  pixel_778  \
0  0.165415  0.121534  0.166468  ...   0.091667   0.118481   0.129612
1  0.143025  0.151176  0.093697  ...   0.143631   0.146712   0.109624
2  0.148633  0.107976  0.132155  ...   0.148414   0.119420   0.176792
3  0.113596  0.120030  0.078028  ...   0.059668   0.157654   0.092070
4  0.104620  0.081641  0.124710  ...   0.058864   0.131078   0.176052

   pixel_779  pixel_780  pixel_781  pixel_782  pixel_783  pixel_784  label
0   0.127776   0.146645   0.120347   0.052042   0.113977   0.117738    0.0
1   0.033783   0.157180   0.142712   0.094203   0.101448   0.113952    1.0
2   0.203287   0.095521   0.095949   0.098923   0.120411   0.087175    0.0
3   0.155091   0.150391   0.106361   0.098669   0.066514   0.039491    1.0
4   0.152258   0.138946   0.102039   0.054295   0.119381   0.059943    0.0

[5 rows x 785 columns]
Training Samples: 1600
Testing Samples: 400
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do r
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 28, 28, 32)	320
batch_normalization (BatchNormalization)	(None, 28, 28, 32)	128
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_4 (Conv2D)	(None, 14, 14, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 14, 14, 64)	256
max_pooling2d_4 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_5 (Conv2D)	(None, 7, 7, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 7, 7, 128)	512
max_pooling2d_5 (MaxPooling2D)	(None, 3, 3, 128)	0
flatten_1 (Flatten)	(None, 1152)	0



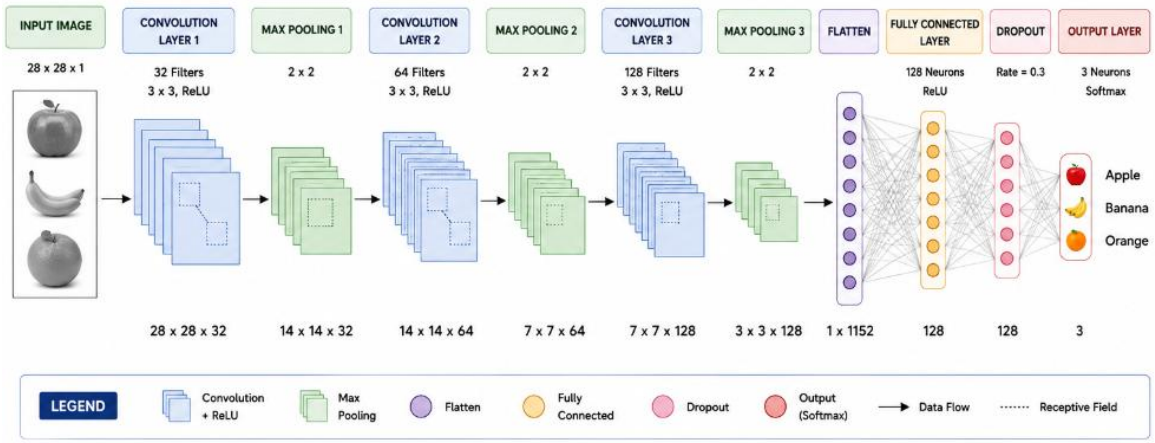


Question 5: Case Study – Image Classification
Answer:

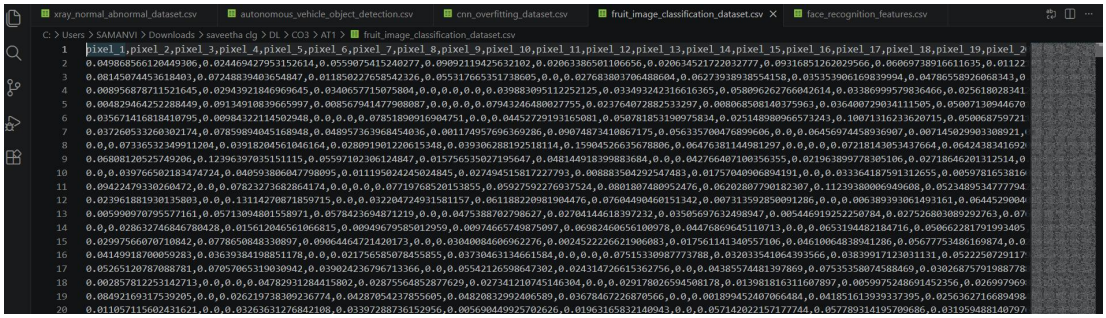
CNN classifies fruits by extracting features such as color, texture, and shape. Pooling reduces the feature size, and the fully connected layer classifies the fruit. The Softmax layer predicts the correct fruit category.

Workflow

Fruit Image
↓
Image Preprocessing
↓
Convolution Layer
↓
ReLU Activation
↓
Max Pooling
↓
Flatten
↓
Fully Connected Layer
↓
Softmax Output
↓
Apple / Banana / Mango / Orange



Dataset:



Program:

```
# ============================================
# FRUIT IMAGE CLASSIFICATION USING CNN
# ============================================

!pip -q install tensorflow

# ----------------------------------------
# Import Libraries
# ----------------------------------------

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import (
    Conv2D,
    MaxPooling2D,
    Flatten,
    Dense,
    Dropout
)

# ----------------------------------------
# Upload Dataset
# ----------------------------------------
```

```
uploaded = files.upload()

# ----------------------------------------
# Load Dataset
# ----------------------------------------

df = pd.read_csv(
    "fruit_image_classification_dataset.csv"
)

print("Dataset Shape:", df.shape)

print("\nFirst 5 Rows:")
print(df.head())

# ----------------------------------------
# Separate Features and Labels
# ----------------------------------------

X = df.iloc[:, :-1].values
y = df["label"].values.astype(int)

# ----------------------------------------
# Reshape Pixels into Images
# ----------------------------------------

X = X.reshape(-1, 28, 28, 1)

# ----------------------------------------
# Train-Test Split
# ----------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# ----------------------------------------
# Build CNN
# ----------------------------------------

model = Sequential()
```

```
# Convolution Block 1

model.add(
    Conv2D(
        32,
        (3,3),
        activation="relu",
        input_shape=(28,28,1)
    )
)

model.add(
    MaxPooling2D((2,2))
)

# ----------------------------------------
# Convolution Block 2
# ----------------------------------------

model.add(
    Conv2D(
        64,
        (3,3),
        activation="relu"
    )
)

model.add(
    MaxPooling2D((2,2))
)

# ----------------------------------------
# Convolution Block 3
# ----------------------------------------

model.add(
    Conv2D(
        128,
        (3,3),
        activation="relu"
    )
)

model.add(
    MaxPooling2D((2,2))
)
```

```
# ----------------------------------------
# Flatten
# ----------------------------------------

model.add(
    Flatten()
)

# ----------------------------------------
# Fully Connected Layer
# ----------------------------------------

model.add(
    Dense(
        128,
        activation="relu"
    )
)

# ----------------------------------------
# Dropout
# ----------------------------------------

model.add(
    Dropout(0.3)
)

# ----------------------------------------
# Output Layer
# ----------------------------------------

model.add(
    Dense(
        3,
        activation="softmax"
    )
)

# ----------------------------------------
# Compile
# ----------------------------------------

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
```

```
    metrics=["accuracy"]
)

# ----------------------------------------
# Display Architecture
# ----------------------------------------

model.summary()

# ----------------------------------------
# Train CNN
# ----------------------------------------

history = model.fit(
    X_train,
    y_train,
    epochs=15,
    batch_size=32,
    validation_split=0.20,
    verbose=1
)

# ----------------------------------------
# Test Model
# ----------------------------------------

loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("\nTest Accuracy:",
      round(accuracy * 100, 2), "%")

print("Test Loss:",
      round(loss, 4))

# ========================================
# ACCURACY GRAPH
# ========================================

plt.figure(figsize=(8,5))

plt.plot(
    history.history["accuracy"],
```

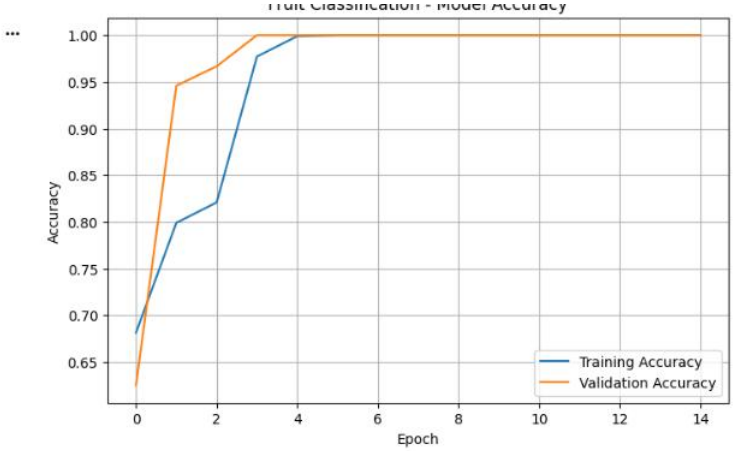
```
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title(
    "Fruit Classification - Model Accuracy"
)

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.legend()

plt.grid()

plt.show()

# ==========================================
# LOSS GRAPH
# ==========================================

plt.figure(figsize=(8,5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title(
    "Fruit Classification - Model Loss"
)

plt.xlabel("Epoch")

plt.ylabel("Loss")
```

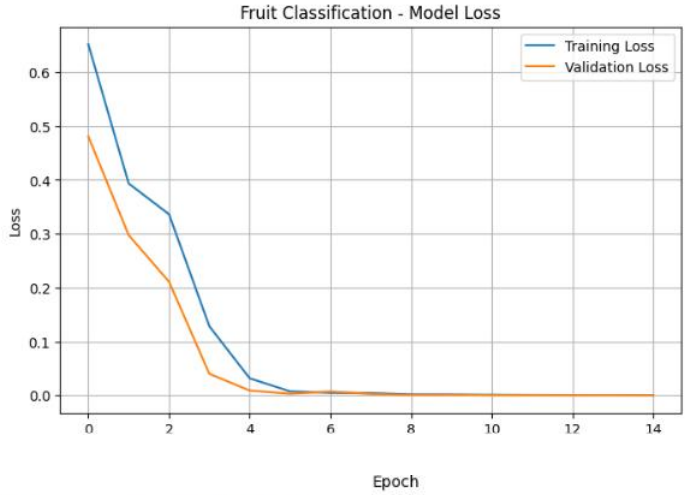
```
plt.legend()

plt.grid()

plt.show()

# ==========================================
# PREDICTION
# ==========================================

predictions = model.predict(X_test)

predicted_classes = np.argmax(
    predictions,
    axis=1
)

# ==========================================
# CONFUSION MATRIX
# ==========================================

cm = confusion_matrix(
    y_test,
    predicted_classes
)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=[
        "Apple",
        "Banana",
        "Orange"
    ]
)

disp.plot()

plt.title(
    "Fruit Classification Confusion Matrix"
)

plt.show()

# ==========================================
# SAMPLE PREDICTIONS
# ==========================================
```

```
class_names = [
    "Apple",
    "Banana",
    "Orange"
]
print("\nSample Predictions:")
for i in range(10):
    actual = class_names[y_test[i]]
    predicted = class_names[
        predicted_classes[i]
    ]
    confidence = (
        np.max(predictions[i]) * 100
    )
    print(
        "Actual:",
        actual,
        "| Predicted:",
        predicted,
        "| Confidence:",
        round(confidence, 2),
        "%"
    )
```

```
   pixel_8   pixel_9  pixel_10  ...  pixel_776  pixel_777  pixel_778  \
0  0.060697  0.011221  0.051782  ...   0.000000   0.000915   0.020899
1  0.035354  0.047866  0.021469  ...   0.007062   0.044139   0.034860
2  0.033493  0.058096  0.033870  ...   0.060884   0.074463   0.107388
3  0.008069  0.036401  0.050071  ...   0.019294   0.063871   0.000000
4  0.050782  0.025149  0.100713  ...   0.051969   0.091327   0.070152

   pixel_779  pixel_780  pixel_781  pixel_782  pixel_783  pixel_784  label
0   0.027023   0.054827   0.037108   0.000000   0.045208   0.054423    0.0
1   0.000000   0.000000   0.044201   0.021155   0.000000   0.000000    0.0
2   0.032790   0.004409   0.000000   0.000000   0.079991   0.000000    0.0
3   0.026036   0.005887   0.047291   0.048802   0.001695   0.001513    0.0
4   0.035499   0.113574   0.004867   0.025988   0.028782   0.025903    0.0

[5 rows x 785 columns]
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_4"
```

Layer (type)	Output Shape	Param #
conv2d_6 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_6 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_7 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_7 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_8 (Conv2D)	(None, 3, 3, 128)	73,856
max_pooling2d_8 (MaxPooling2D)	(None, 1, 1, 128)	0
flatten_2 (Flatten)	(None, 128)	0
dense_15 (Dense)	(None, 128)	16,512
dropout_2 (Dropout)	(None, 128)	0
dense_16 (Dense)	(None, 3)	387






```
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step
```